

# Shared Foundations

The session log, the Prisma singleton, session ownership and shared types

---

EdgePilot AI - Team 15  
Documentation set, 23 August 2026  
5 documents in this volume

## src/core/ - cross-cutting concerns

---

**Work package:** benchmark and provider layer.

### What this folder is

---

Things used by more than one module and owned by none of them. Exactly one concern lives here now: the session log.

### What happened here

---

Two whole subfolders were removed and one file was replaced.

rate-limit/ and security/ guarded endpoints that have no authentication and no public URL. Security machinery that protects nothing is not neutral - it still has to be read, maintained and reasoned about by everyone who comes after, and it implies a threat model the application does not actually have.

logging/logger.ts was replaced by logging/SessionLog.ts, which is the difference between a logging utility and a logging **policy**: a redaction rule, an export format, and opt-in behaviour.

### Subfolders

---

| Subfolder | What it holds                          |
|-----------|----------------------------------------|
| logging/  | The session activity log and its store |

### Connected folders

---

- src/app/api/v1/session-log/ - the endpoints that expose it.
- tests/core/ - the suite.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- src/

## src/lib/ - process-level singletons

**Work package:** integration and API surface; `sessionOwner.ts` belongs to the benchmark layer.

### What this folder is

Two files, both answering a question that has exactly one correct answer per process: which Prisma client, and who owns this row.

### What happened here

`sessionOwner.ts` is new, and it closed a real data leak.

Two routes previously attributed every row they created to a single hardcoded user, `local-sandbox@edgepilot.invalid`, with a comment saying session handling did not exist yet. On one laptop that is harmless. The moment the application is hosted it is a leak: GET would hand every visitor every other visitor's workloads, because they would all be the same user.

The replacement **fails closed**. A request with no session id gets a 400. It does not fall back to a shared user - that fallback is exactly the leak, and it fails silently, because the caller gets a 200 and someone else's data.

### Files

| File                         | What it is                                                                                                                                                                                                   |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>prisma.ts</code>       | 33 lines. The single <code>PrismaClient</code> for the process, cached on <code>globalThis</code> . Without the cache, a new client per hot reload exhausts the database connection limit within a few edits |
| <code>sessionOwner.ts</code> | 76 lines. <code>sessionIdOf(request)</code> , <code>resolveOwnerId(sessionId)</code> which upserts a <code>User</code> , and <code>missingSession()</code> which returns the 400                             |

### Connected folders

- `src/components/dashboard/` - `session.ts`, the browser half that generates the id.
- `src/app/api/v1/workloads/` - the route that used to attribute everything to one user.
- `prisma/` - the `users.session_id` column and the migration that added it.

### Referenced from

These folders point here. Each link below resolves in both directions.

- `docs/benchmark/`
- `src/`
- `src/app/api/v1/`
- `src/modules/benchmark/infrastructure/repositories/`

## src/shared/ - types crossing module boundaries

---

**Work package:** integration and API surface.

### What this folder is

---

The scaffold's shared type definitions and zod schemas.

### What happened here

---

types/device.ts was removed with the device module. The rest is largely unchanged, and its role has narrowed in a way worth understanding: the benchmark module now defines its own richer measurement envelope in application/dtos/BenchmarkMeasurement.ts, and the dashboard imports **that** rather than these. These remain the scaffold contract and the shape the original endpoints were written against.

### Subfolders

---

| Subfolder | What it holds                                                 |
|-----------|---------------------------------------------------------------|
| types/    | Benchmark, provider and workload types with their zod schemas |

### Connected folders

---

- src/modules/benchmark/application/dtos/ - the superset that supersedes these for measurement.
- src/app/api/v1/ - the routes that validate with them.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- src/

## src/core/logging/ - the session activity log

**Work package:** benchmark and provider layer.

### What this folder is

A per-session record of what the application did, exportable as JSON, with no account and no user identity attached.

### What happened here

New, replacing a generic `logger.ts`. Two rules are built in rather than configurable, and both are there because the obvious design would have been wrong.

**Redaction is not optional.** The obvious thing for a benchmark log to record is the prompt. The prompt is also the confidential thing this product exists to warn people about - the egress warning tells a user "this text is about to leave your machine", and a log that then writes the same text into a file they can email around has defeated the warning it just issued. So prompt **content** is never recorded by default. Only a length and a short digest, which is enough to prove two runs used the same prompt without disclosing what it was.

**Logging is opt-in by header.** A request without `x-edgepilot-session` is not logged anonymously and not logged to a default bucket - it is not logged at all. That keeps the default behaviour "we kept no record of what you ran", which is the only default consistent with the rest of the application.

The store is cached on `globalThis` for the same reason the Prisma client is: Next.js hot-reloads modules in development, and a fresh store per reload would drop every session log mid-use.

### Files

| File                            | What it is                                                                                                                                                                                            |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SessionLog.ts</code>      | 407 lines. The log, its event and category vocabulary, <code>redact</code> , <code>digestPrompt</code> , the in-memory store, <code>buildSharePayload</code> and <code>SHARE_CONSENT_STATEMENT</code> |
| <code>sessionLogStore.ts</code> | 56 lines. The process-wide store, the <code>x-edgepilot-session</code> header name, and <code>logForRequest</code>                                                                                    |

### Connected folders

- `src/app/api/v1/session-log/` and `share/` - the endpoints.
- `src/components/dashboard/` - `session.ts` generates the header value; `api.ts` writes the header name out as a literal rather than importing it, because importing this module from the client would instantiate server state at module scope.
- `src/modules/benchmark/core/services/` - `egress-warning.ts`, the warning this log must not undermine.
- `tests/core/` - `session-log.test.ts`.

### Referenced from

These folders point here. Each link below resolves in both directions.

- src/components/history/
- src/core/

## src/shared/types/ - the scaffold contract

---

**Work package:** integration and API surface.

### What this folder is

---

Four small files pairing a TypeScript type with the zod schema that validates it, so validation and typing cannot disagree.

### What happened here

---

`device.ts` was deleted with the device module, and `index.ts` lost its export of it - a one-line change that is the visible end of a removal touching the UI, the API, the module and the database.

`benchmark.ts` changed by three lines. That small number is deliberate: the benchmark module's own DTO is a **superset** of this shape rather than a replacement, keeping every field it already had, in the same place, with the same name, so the dashboard being written against this contract did not have to be rewritten.

### Files

---

| File                      | What it is                                                                                                                  |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>benchmark.ts</code> | 46 lines. <code>BenchmarkRequest</code> , <code>BenchmarkResult</code> , <code>ReadinessScore</code> and their schemas      |
| <code>provider.ts</code>  | 46 lines. <code>Provider</code> , <code>AIPProvider</code> , <code>AIResponse</code> , <code>ProviderBenchmarkResult</code> |
| <code>workload.ts</code>  | 17 lines. <code>Workload</code> , <code>CreateWorkloadRequest</code> and their schemas                                      |
| <code>index.ts</code>     | 4 lines. Re-exports. Three lines now, where there were four                                                                 |

### Connected folders

---

- `src/modules/benchmark/application/dtos/` - the superset envelope.
- `src/app/api/v1/workloads/` - validates against `CreateWorkloadSchema`.
- `src/shared/` - the parent.